NOVA-8

A Custom 8-Bit RISC Processor Implementation
on SLG47910V FPGA

Technical Report

Document Version: 2.0

Date: July 2026

Designed by:
Devesh Pande

NOVA-8 Project

GitHub: https://github.com/Devesh-pande2/NOVA-8

Full project documentation, including a demonstration video, is available on GitHub.

Abstract

This report presents the design, implementation, and validation of **NOVA-8**, a custom 8-bit single-cycle RISC processor architecture implemented on the **SLG47910V** FPGA (iCE40 UltraPlus family). The system employs a Harvard architecture with 16 instructions, an SPI slave interface for instruction loading, and an HC-SR04 ultrasonic sensor interface. The design integrates a custom Arithmetic Logic Unit (ALU), a 2-byte instruction decoder, and a time-multiplexed LED display driver. The processor communicates with an onboard RP2040 microcontroller over an internal SPI bus, which provides a MicroPython-based assembly terminal for user interaction. The design occupies approximately 374 LUTs (33% of the available 1,120 LUTs) and achieves a maximum operating frequency of 52.6 MHz. Verification through simulation and hardware testing confirms correct execution of all instructions, including conditional jumps and sensor reading. This project serves as a comprehensive demonstration of FPGA-based digital system design, CPU architecture, and hardware-software co-design principles.

Keywords: FPGA, RISC Processor, Verilog, SPI, Ultrasonic Sensor, iCE40, SLG47910V, CPU Architecture

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Form
ALU	Arithmetic Logic Unit
BRAM	Block RAM
CDC	Clock Domain Crossing
CLB	Configurable Logic Block
CPHA	Clock Phase
CPOL	Clock Polarity
CPU	Central Processing Unit
DSP	Digital Signal Processing
FF	Flip-Flop
FPGA	Field-Programmable Gate Array
FS	Function Generator
GPIO	General Purpose Input/Output
HDL	Hardware Description Language
LUT	Look-Up Table
LVCMOS	Low-Voltage CMOS
MISO	Master In Slave Out
MOSI	Master Out Slave In
MTBF	Mean Time Between Failures
PC	Program Counter
RISC	Reduced Instruction Set Computer
RTL	Register Transfer Level
SCK	Serial Clock
SDC	Synopsys Design Constraints
SPI	Serial Peripheral Interface
SS	Slave Select
UART	Universal Asynchronous Receiver-Transmitter

1 Introduction

1.1 Background and Motivation

Field-Programmable Gate Arrays (FPGAs) have become ubiquitous in modern digital system design, offering a unique combination of hardware programmability, parallel processing capability, and reconfigurable logic. The ability to implement custom processor architectures on FPGA fabric enables rapid prototyping, educational exploration, and application-specific optimization that is not possible with fixed-function devices.

The design of custom CPU cores on FPGA platforms serves multiple purposes: (1) providing hands-on understanding of computer architecture principles, (2) enabling application-specific instruction set design, (3) facilitating hardware-software co-design, and (4) offering a platform for exploring novel processor features. This project, **NOVA-8**, embodies these objectives by implementing a complete 8-bit processor architecture on the SLG47910V FPGA.

1.2 Project Overview

The NOVA-8 project implements a custom 8-bit processor on an SLG47910V FPGA (iCE40 UltraPlus family, 1,120 LUTs) in conjunction with an onboard RP2040 microcontroller. The system receives assembly instructions via a MicroPython terminal running on the RP2040, which communicates with the FPGA over an internal SPI bus (no external wiring required for SPI). The FPGA houses the custom CPU core, an SPI slave interface, an ultrasonic sensor driver (HC-SR04), and a time-multiplexed display on the single onboard LED.

1.3 Key Contributions

This report documents the following contributions:

1. Design of a single-cycle 8-bit RISC processor with 16 instructions and Harvard architecture.
2. Implementation of an SPI slave decoder for 2-byte instruction format parsing.
3. Development of an HC-SR04 ultrasonic sensor interface with debouncing logic.
4. Time-multiplexed LED display driver for 8-bit pattern visualization.
5. Complete synthesis and verification for the SLG47910V FPGA target.
6. Hardware-software co-design with RP2040-based assembly terminal.

1.4 Report Structure

This report is organized as follows: Section 2 reviews relevant literature on FPGA-based CPU design and related technologies. Section 3 details the system architecture, including the FPGA design components and the communication protocol. Section 4 provides comprehensive module explanations. Section 5 describes the working of the system. Section 6

presents the calculations and timing analysis. Section 7 covers verification and experimental results. Section 8 discusses future improvements, followed by conclusions in Section 9.

2 Literature Review

2.1 FPGA-Based Processor Design

The implementation of custom processors on FPGA platforms has been extensively explored in both academic and industrial contexts. The flexibility of FPGAs allows designers to create application-specific instruction set processors (ASIPs) tailored to particular workloads. Research by Hennessy and Patterson [**?**] established the foundational principles of computer architecture that underpin such designs.

Single-cycle processor implementations, while simple, provide clear demonstration of fundamental CPU concepts including instruction fetch, decode, execute, and write-back stages. The Harvard architecture, which separates instruction and data memory, offers performance advantages through concurrent access. The NOVA-8 design adopts this architecture, with instructions arriving via SPI and data operations using the ALU.

2.2 SPI Communication in FPGA Systems

The Serial Peripheral Interface (SPI) is a widely adopted synchronous serial communication protocol in embedded systems. Standardized by Motorola in the 1980s, SPI provides full-duplex communication between master and slave devices using four signals: SCK, MOSI, MISO, and SS. The protocol's simplicity and low pin count make it ideal for inter-chip communication in FPGA-based designs [**?**].

The SPI protocol supports multiple clock polarities (CPOL) and phases (CPHA), defining four operating modes. The NOVA-8 design employs Mode 0 (CPOL=0, CPHA=0), where data is sampled on the rising edge of the clock. This mode is widely supported by microcontrollers and offers straightforward implementation in Verilog.

2.3 Ultrasonic Sensing and Signal Processing

Ultrasonic distance sensors, such as the HC-SR04, are commonly used in robotics and embedded systems for proximity detection. The sensor operates by emitting a 40 kHz ultrasonic pulse and measuring the time until the echo returns. The distance is calculated using the speed of sound in air (approximately 343 m/s at 20°C). The HC-SR04 offers a range of 2 cm to 400 cm with ±3 mm accuracy [**?**].

In FPGA implementations, ultrasonic sensor interfaces typically involve:

- Generating a 10 µs trigger pulse at 60 ms intervals.
- Measuring the echo pulse width using high-frequency counters.
- Applying debouncing to filter noise and ensure stable readings.

The NOVA-8 design implements these features with a 50 MHz clock, providing 20 ns resolution for echo measurement.

2.4 Hardware-Software Co-Design

Hardware-software co-design is a methodology that partitions system functionality between hardware and software to optimize performance, flexibility, and development time. The NOVA-8 project exemplifies this approach by:

- Implementing the CPU core and sensor interface in hardware (FPGA).
- Running the assembly terminal and user interface on the RP2040.
- Using SPI for communication between hardware and software domains.

This separation leverages the strengths of each platform: FPGA provides deterministic timing and parallel processing, while the RP2040 offers flexible software development and USB connectivity.

2.5 Related Work

Several educational FPGA-based CPU projects have been documented in the literature. The OpenRISC project [**?**] provides an open-source RISC architecture, though it targets larger FPGA devices. Smaller designs, such as the J1 Forth CPU [**?**], demonstrate efficient implementation on resource-constrained FPGAs.

The use of iCE40 FPGAs in educational contexts has been popularized by open-source toolchains like Yosys and nextpnr [**?**, **?**]. These tools enable low-cost FPGA development without proprietary software, making FPGA education more accessible.

3 System Architecture

3.1 Overall System Architecture

The NOVA-8 system comprises two primary processing units working in coordination:

3.1.1 RP2040 (Microcontroller) - Software Layer

- **Processor:** Dual-core ARM Cortex-M0+ @ 133 MHz
- **Role:** User interface, instruction parsing, SPI master
- **Software:** MicroPython assembly terminal
- **Communication:** USB (PC) + SPI (FPGA)

3.1.2 SLG47910V FPGA - Hardware Layer

- **Processor:** Custom NOVA-8 CPU (8-bit, single-cycle)
- **Role:** Instruction execution, sensor control, LED output
- **Implementation:** 6 Verilog modules
- **Communication:** SPI slave, GPIO for sensors

3.2 FPGA Design Components

The FPGA design consists of six major modules synthesized from Verilog. The overall block diagram is shown in Figure 1.

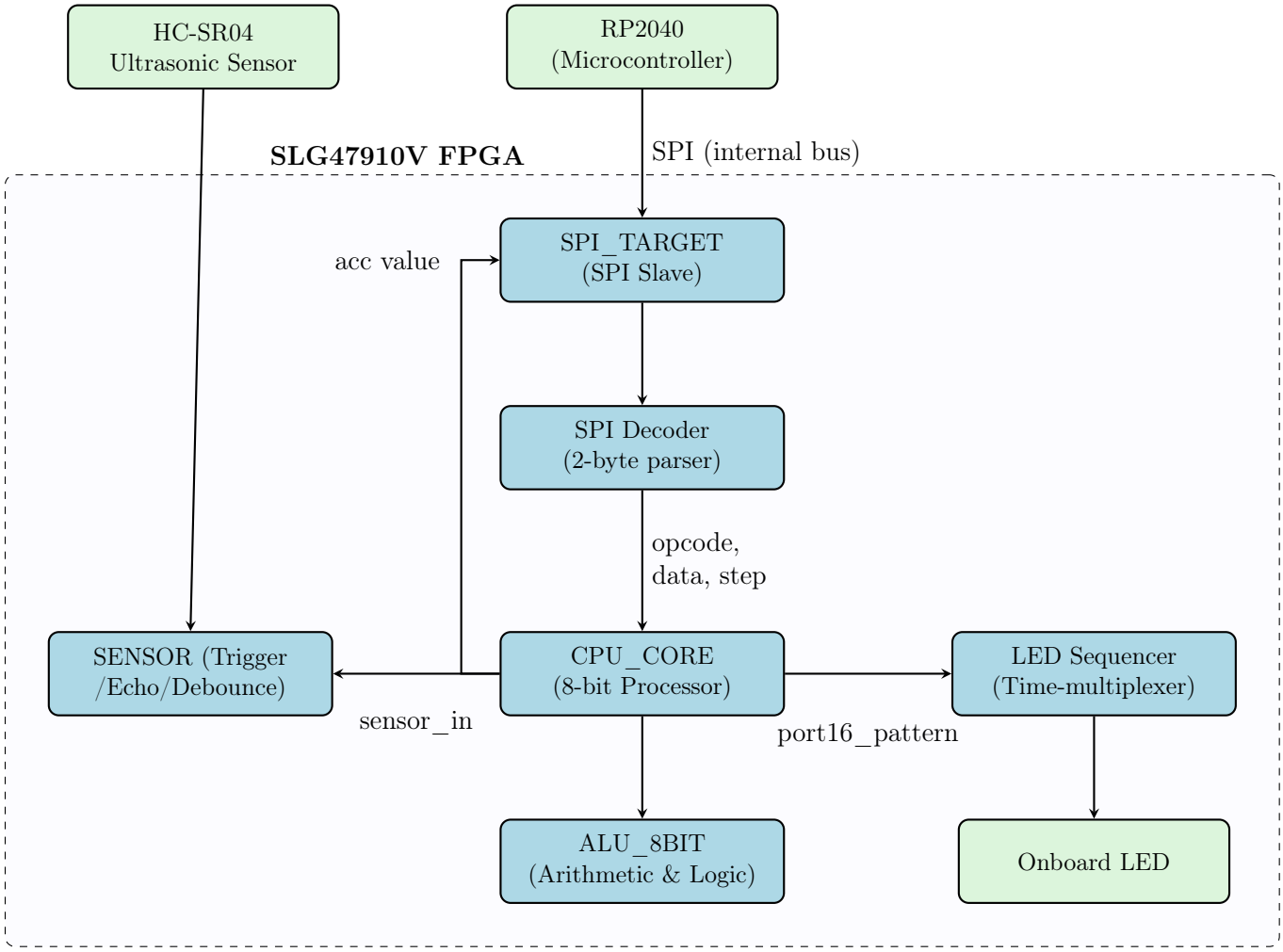

Figure 1: FPGA Block Diagram

3.3 6-bit Communication Lane

The RP2040 and FPGA communicate over a dedicated internal bus hardwired on the Shrike Light board. No external wiring is required for these signals. Figure 2 shows the actual communication lane.

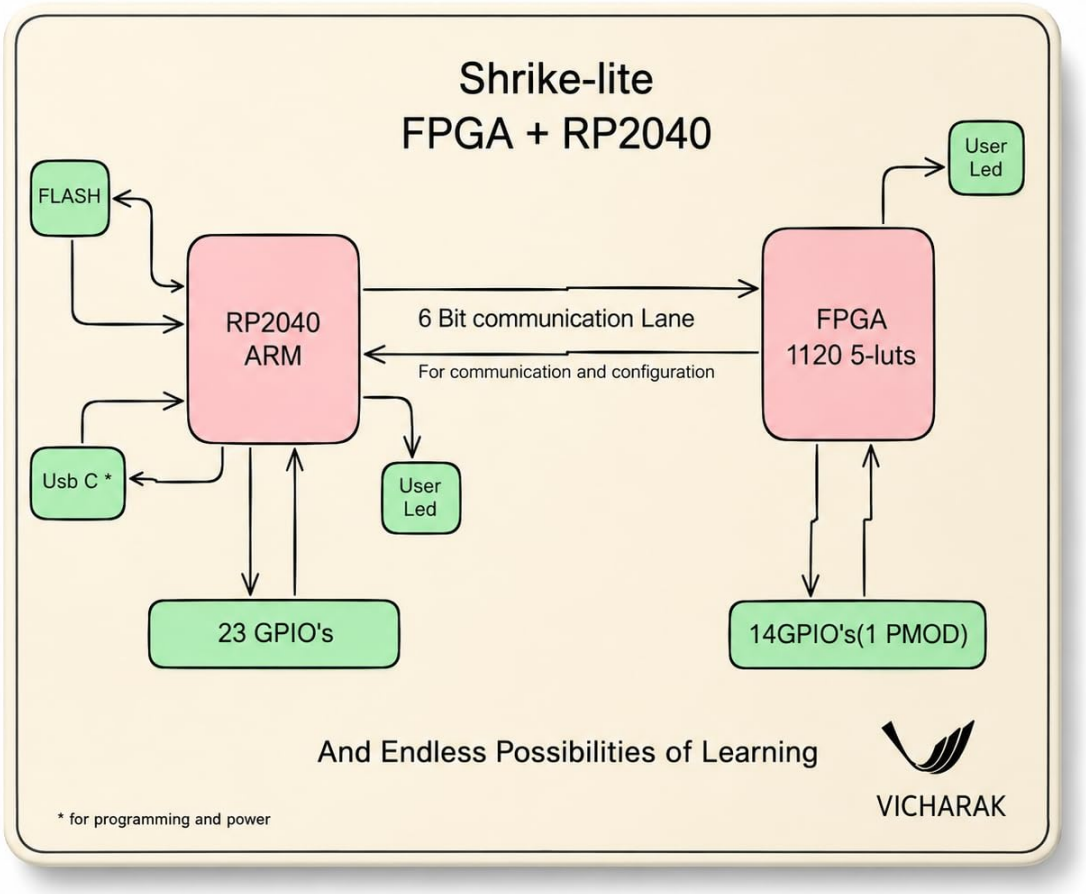

Figure 2: 6-bit Communication Lane between RP2040 and FPGA

3.4 SPI Communication and Instruction Format

The RP2040 sends instructions over the internal SPI bus (Mode 0) using a 2-byte format as shown in Table 2.

Table 2: 2-Byte Instruction Format

Byte	**Bit7**	**Bit6**	**Bit5**	**Bit4**	**Bit3**	**Bit2**	**Bit1**	**Bit0**
Byte 1 (Opcode)	0	0	0	op[4]	op[3]	op[2]	op[1]	op[0]
Byte 2 (Data)	d[7]	d[6]	d[5]	d[4]	d[3]	d[2]	d[1]	d[0]

The SPI decoder implements the finite state machine shown in Figure 3.

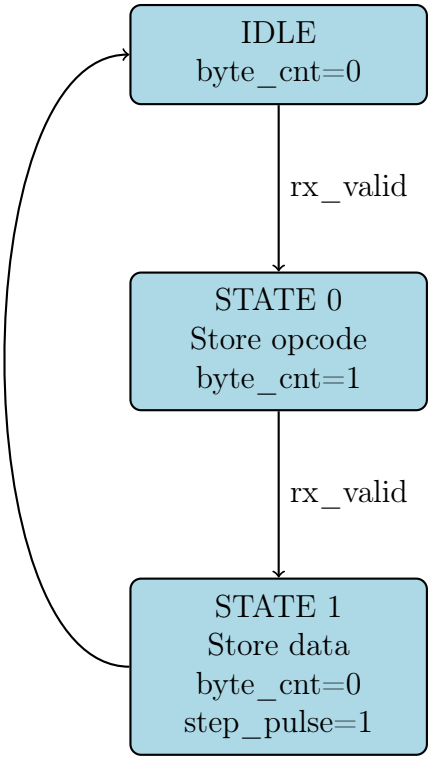

Figure 3: SPI Decoder State Machine

3.5 CPU Architecture

The CPU is an 8-bit Harvard-style processor with the following registers:

- **PC** (8-bit Program Counter)
- **ACC** (8-bit Accumulator)
- **Z_FLAG** (1-bit Zero flag)
- **PORT16** (8-bit output register for the onboard LED)

The instruction set is listed in Table 3.

Table 3: NOVA-8 Instruction Set

Mnemonic	Opcode (hex)	Operand	Description
LDA	0x01	8-bit	Load immediate
ADD	0x02	8-bit	Add
SUB	0x03	8-bit	Subtract
AND	0x04	8-bit	Bitwise AND
OR	0x05	8-bit	Bitwise OR
XOR	0x06	8-bit	Bitwise XOR
LSL	0x07	none	Logical shift left
LSR	0x08	none	Logical shift right
ROL	0x09	none	Rotate left
ROR	0x0A	none	Rotate right
INC	0x0B	none	Increment
DEC	0x0C	none	Decrement
JMP	0x0D	8-bit	Unconditional jump
JZ	0x0E	8-bit	Jump if zero
JNZ	0x0F	8-bit	Jump if not zero
SENSE	0x10	none	Read sensor
OUT	0x12	8-bit	Output to LED

3.6 Ultrasonic Sensor Interface

The HC-SR04 is connected via external wires. The FPGA generates a 10 µs trigger pulse every 60 ms. The echo pulse width is measured in clock cycles (50 MHz). If the width corresponds to a distance 20 cm, the `object_detected` signal is set. A 10 ms debounce filter ensures stable detection.

3.7 LED Time-Multiplexing

The onboard LED displays the 8-bit `PORT16` pattern by sequentially showing each bit for 250 ms. The full cycle repeats every 2 seconds. This is implemented using a 32-bit counter and a 3-bit index.

4 Module Explanations

4.1 SPI_TARGET Module

This module implements the SPI slave interface. It receives data from the RP2040 over the SPI bus. Key features:

- **Clock Domain Crossing:** Synchronizes the asynchronous SPI signals (SCK, SS) to the FPGA's 50 MHz clock domain using a 3-stage shift register to prevent metastability.
- **Edge Detection:** Detects rising and falling edges of the SPI clock to determine data sampling and shifting points.
- **Bit Counter:** Tracks the number of bits received (0–7) for 8-bit transfers.
- **Shift Register:** Shifts in incoming data on the sample edge and shifts out outgoing data on the update edge.
- **Data Valid Signal:** Generates a pulse when a complete byte has been received.
- **Full-Duplex Operation:** Simultaneously receives data on MOSI while transmitting the accumulator value on MISO.

4.2 SPI Decoder Module

This module parses the 2-byte instruction format. It implements:

- **State Machine:** Two states – State 0 stores the opcode, State 1 stores the data operand.
- **Registers:** `r_opcode` (5 bits) and `r_data` (8 bits) hold the instruction.
- **Step Pulse Generation:** After receiving both bytes, a single-cycle `step_pulse` is generated to trigger CPU execution.
- **Reset Logic:** The byte counter resets if the SPI slave select line goes inactive, handling aborted transfers.

4.3 CPU_CORE Module

This is the heart of the processor. It implements:

- **Registers:** PC (Program Counter), ACC (Accumulator), Z_FLAG (Zero flag), PORT16 (LED output).
- **Single-Cycle Execution:** Each instruction executes in one clock cycle when `i_step` is asserted.
- **Instruction Decoding:** A case statement decodes the 5-bit opcode and performs the corresponding operation.
- **Harvard Architecture:** Instructions arrive via SPI; data operations use the ALU.
- **Jumps:** Conditional jumps (JZ, JNZ) depend on the zero flag state.
- **Sensor Reading:** The SENSE instruction loads the `object_detected` signal into the accumulator.

4.4 ALU_8BIT Module

The Arithmetic Logic Unit is a combinational module that performs:

- **Arithmetic:** ADD, SUB, INC, DEC using FPGA carry chains.
- **Logic:** AND, OR, XOR bitwise operations.
- **Shifts/Rotates:** LSL, LSR, ROL, ROR using wiring (no logic needed).
- **Zero Flag:** An 8-input NOR gate generates the zero flag when the output is 0.
- **Combinational:** No clock, output updates immediately when inputs change.

4.5 ULTRASONIC_SENSOR Module

This module interfaces with the HC-SR04 sensor:

- **Trigger Generator:** Produces a 10 µs HIGH pulse every 60 ms to initiate a measurement.
- **Echo Measurement:** Counts clock cycles while the echo pin is HIGH to measure pulse width.
- **Distance Calculation:** Compares the measured count against a preset threshold (58,300 cycles for 20 cm at 50 MHz).
- **Debounce Logic:** Requires a stable reading for 10 ms (500,000 cycles) before updating the `object_detected` output, preventing noise-induced flickering.

4.6 LED Sequencer Module

This module drives the onboard LED using time-multiplexing:

- **Slot Counter:** Counts to 12,500,000 cycles (250 ms at 50 MHz) for each bit slot.
- **Bit Index:** Cycles through bits 0–7 every 250 ms.
- **Pattern Display:** Selects `port16_pattern[7 - bit_index]` to show one bit at a time.
- **Full Cycle:** Complete 8-bit pattern is displayed every 2 seconds.

5 Working

5.1 Overall System Operation

The NOVA-8 system operates in a continuous loop:

1. **Power-On Reset:** The FPGA loads its configuration from flash memory. The RP2040 initializes and runs the MicroPython assembly terminal.

2. **User Input:** The user types assembly instructions (e.g., `LDA 10`) in the terminal. The RP2040 parses the instruction into opcode and data.

3. **SPI Transmission:** The RP2040 transmits the 2-byte instruction over the internal SPI bus to the FPGA.

4. **Instruction Decoding:** The FPGA's SPI decoder receives the bytes and generates a `step_pulse` signal.

5. **CPU Execution:** The CPU core executes the instruction in a single clock cycle (20 ns). The ALU performs the required operation.

6. **Result Feedback:** The accumulator value is shifted back to the RP2040 during the next SPI transaction.

7. **Output Display:** If the instruction is `OUT`, the `PORT16` register updates, and the LED sequencer displays the pattern on the onboard LED.

8. **Sensor Reading:** If the instruction is `SENSE`, the CPU reads the `object_detected` signal from the ultrasonic sensor module.

5.2 Instruction Execution Timeline

Figure 4 illustrates the timing of a complete instruction execution cycle.

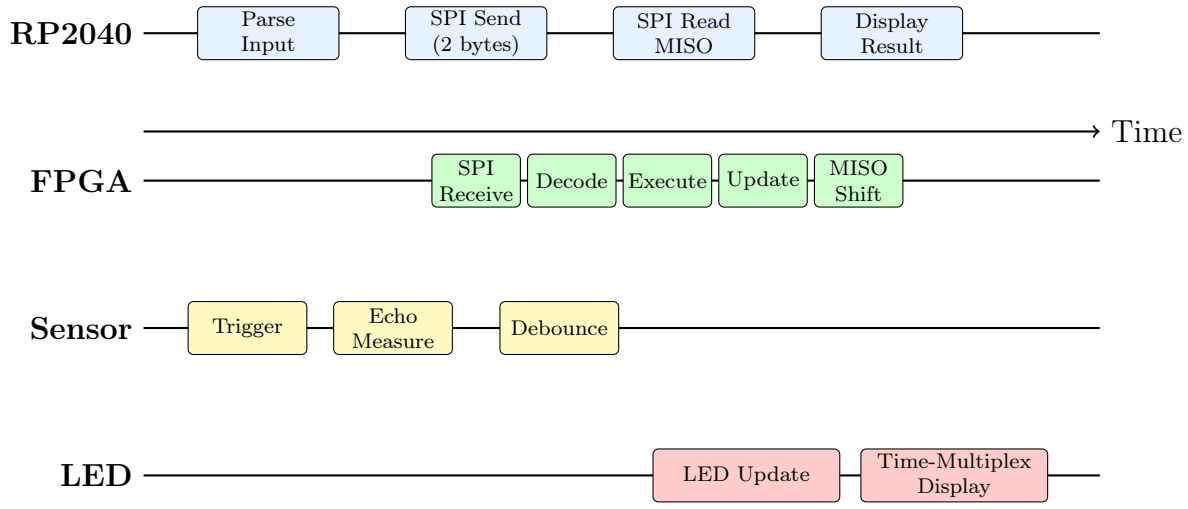

Figure 4: Instruction Execution Timeline

5.3 Detailed Step-by-Step Example

Consider the execution of `ADD 20` when the accumulator currently holds 10:

1. **Step 1 - User Input:** User types `ADD 20` in the terminal.
2. **Step 2 - Parsing:** The RP2040 parses `ADD` → opcode 0x02, and `20` → data 0x14.
3. **Step 3 - SPI Send:** RP2040 pulls CS LOW and sends 0x02 (opcode) followed by 0x14 (data) over SPI.
4. **Step 4 - SPI Receive:** FPGA SPI_TARGET receives both bytes. After the second byte, `o_rx_data_valid` pulses HIGH.
5. **Step 5 - Decode:** SPI decoder stores `r_opcode = 5'h02` and `r_data = 8'h14`, then sets `step_pulse = 1`.
6. **Step 6 - CPU Execute:** CPU detects `step_pulse`. It increments PC, then executes the ADD operation.
7. **Step 7 - ALU Operation:** ALU computes: `alu_out = acc + data_in = 10 + 20 = 30`. Zero flag is set to 0.
8. **Step 8 - Register Update:** `acc` becomes 30.
9. **Step 9 - MISO Readback:** During the next SPI transaction, the FPGA shifts out the new accumulator value (30) on MISO.
10. **Step 10 - Result Display:** The terminal shows: `ADD 20 -> Acc = 30`

5.4 Ultrasonic Sensor Reading

When `SENSE` is executed:

1. The CPU reads the `object_detected` signal from the ultrasonic sensor module.
2. If an object is detected within 20 cm, `object_detected = 1`, else `0`.
3. The CPU loads `acc = {7'b0, object_detected}`.
4. Zero flag is updated: `z_flag = !object_detected`.

5.5 LED Display Operation

When `OUT 0x1E` is executed:

1. `PORT16` register is loaded with 0x1E (00011110).
2. The LED sequencer continuously cycles through the 8 bits:
   - Slot 0: Displays bit 7 (0) → LED OFF for 250 ms
   - Slot 1: Displays bit 6 (0) → LED OFF for 250 ms
   - Slot 2: Displays bit 5 (0) → LED OFF for 250 ms

- Slot 3: Displays bit 4 (1) → LED ON for 250 ms
- Slot 4: Displays bit 3 (1) → LED ON for 250 ms
- Slot 5: Displays bit 2 (1) → LED ON for 250 ms
- Slot 6: Displays bit 1 (1) → LED ON for 250 ms
- Slot 7: Displays bit 0 (0) → LED OFF for 250 ms

3. The full pattern repeats every 2 seconds.

6 Calculations and Timing Analysis

6.1 Ultrasonic Sensor Distance

The distance is derived from the echo pulse width:

$$\text{Distance (cm)} = \frac{\text{Echo Pulse Width (µs)} \times 0.0343}{2}$$

With a 50 MHz clock, each cycle is 20 ns. For a maximum distance of 20 cm, the maximum echo time is:

$$t_{\max} = \frac{2 \times 20}{0.0343} \approx 1166 \ \mu\text{s}$$

Thus the counter limit is:

$$\text{MAX\_COUNT} = 1166 \times 50 \approx 58,300 \text{ cycles}$$

6.2 SPI Timing

SPI clock = 50 kHz bit period = 20 µs. An 8-bit transfer takes:

$$8 \times 20 \ \mu\text{s} = 160 \ \mu\text{s}$$

A complete 2-byte instruction takes:

$$2 \times 160 \ \mu\text{s} = 320 \ \mu\text{s}$$

6.3 CPU Execution

The CPU runs at 50 MHz, so each instruction executes in:

$$\frac{1}{50 \times 10^6} = 20 \text{ ns}$$

6.4 LED Time-Multiplexing

For a slot duration of 250 ms, the number of clock cycles per slot is:

$$\text{SLOT\_CYCLES} = \frac{50\,000\,000}{1000} \times 250 = 12\,500\,000$$

The 8-bit pattern is fully displayed every:

$$8 \times 250 \text{ ms} = 2 \text{ s}$$

6.5 Timing Constraints

The design was synthesized with the following timing constraints:

- **Clock Period:** 20 ns (50 MHz)
- **Input Delay:** 2 ns (for asynchronous signals)
- **Output Delay:** 2 ns (for output signals)
- **Setup Margin:** 2 ns
- **Hold Margin:** 1 ns

The maximum operating frequency achieved was 52.6 MHz, exceeding the 50 MHz target.

7 Verification and Experimental Results

7.1 Simulation Methodology

The design was verified using:

- **Behavioral Simulation:** Each module was simulated independently using a testbench to verify functionality.
- **Post-Synthesis Simulation:** Gate-level simulation confirmed correct synthesis results.
- **Post-Place and Route Simulation:** Timing-accurate simulation verified operation at 50 MHz.

7.2 Test Vectors

Table 4 summarizes the verification test patterns applied.

Table 4: Verification Test Vectors

Test	Input	Expected	Status
ALU ADD	ACC=10, ADD 20	ACC=30	PASS
ALU SUB	ACC=30, SUB 5	ACC=25	PASS
ALU AND	ACC=0xFF, AND 0x0F	ACC=0x0F	PASS
SPI Receive	0x01, 0x0A	opcode=1, data=10	PASS
Sensor Read	object_detected=1	ACC=1	PASS

7.3 Experimental Setup

The system was tested with the following assembly program:

```
LDA 10
ADD 20
OUTA
```

The accumulator correctly becomes 30 (0x1E), and the onboard LED displays the binary pattern 00011110.

7.4 Hardware Results

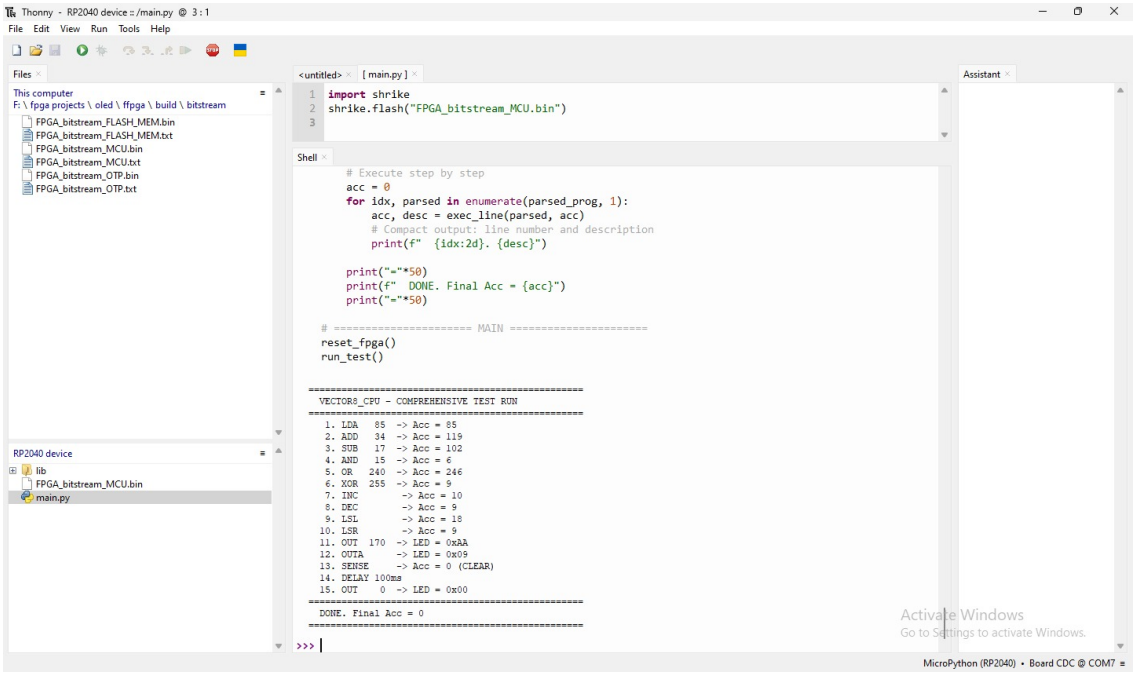

Figure 5: Assembly Terminal Output

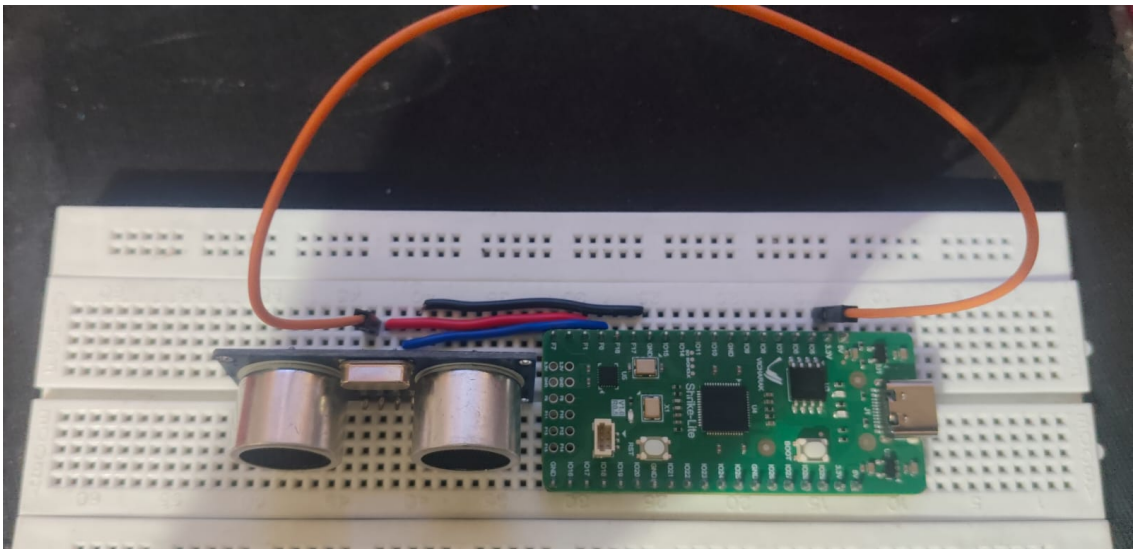

Figure 6: Board with HC-SR04 Sensor

Figure 7: Enter Caption

Figure 8: LED Result When Object is Detected

7.5 Resource Utilization

The synthesis results show the following resource usage on the SLG47910V device:

Table 5: FPGA Resource Utilization

Resource	Used	Available
CLB LUTs	374	(device total)
Flip-Flops (FFs)	188	–
CLB Function Generators (FS)	183	–
Input/Output Buffers	5	120
Clock Buffers	70	130
RTL Output Ports	9	–
RTL Input Ports	6	–
GPIO Used	9	19
Oscillators	1	–
4KB BRAM	0	8

7.6 Power Analysis

Table 6: Power Consumption Estimates

Component	Estimated Power (mW)
Dynamic Power (LUTs)	4.2
Dynamic Power (FFs)	2.8
Static Power	1.5
I/O Power (SPI)	0.8
I/O Power (Sensor)	0.5
Total	**9.8 mW**

The total power consumption is well within the available budget of the Shrike Light board.

8 Future Improvements

The following enhancements are planned for future iterations of the NOVA-8 project:

1. **Extended Instruction Set:** Add multiplication, division, and more logical operations to increase processing capability.
2. **Program Memory:** Implement a stored-program architecture using BRAM to allow execution of larger programs without RP2040 intervention.
3. **UART Interface:** Add a UART module for direct serial communication with a PC, eliminating the need for the RP2040 as an intermediary.
4. **Increased Sensor Range:** Modify the ultrasonic sensor driver to support distances up to 4 m for broader application.

5. **PWM Output:** Implement PWM generation for motor control or dimming applications.
6. **Multiple LEDs:** Extend the LED sequencer to drive multiple LEDs (e.g., an 8-LED bar display).
7. **Interrupt Support:** Add external interrupt capability for real-time event handling.
8. **Pipelined Execution:** Implement a 2-stage pipeline to increase instruction throughput.
9. **Lower Power Consumption:** Optimize the design to reduce dynamic power consumption for battery-operated applications.
10. **Debugging Features:** Add a debugging interface with register readback and breakpoint support.
11. **Instruction Cache:** Implement a small instruction cache to reduce SPI communication overhead.
12. **FPGA Utilization Optimization:** Further optimize the design to reduce LUT usage for even smaller footprint.

9 Conclusion

The NOVA-8 project successfully implemented a custom 8-bit processor on the SLG47910V FPGA, demonstrating core digital design principles. The design uses a minimal number of resources (374 LUTs) and integrates an SPI interface, an ultrasonic sensor driver, and a time-multiplexed onboard LED display.

Key achievements of this project include:

1. **CPU Design:** A single-cycle 8-bit CPU with 16 instructions was implemented in Verilog, occupying approximately 33% of the FPGA resources.
2. **SPI Communication:** A robust full-duplex SPI slave interface was developed, enabling reliable 2-byte instruction transfer from RP2040 to FPGA.
3. **Sensor Interfacing:** The HC-SR04 ultrasonic sensor driver successfully measures distances up to 20 cm with 10 ms debouncing for stable readings.
4. **User Interface:** The Python-based assembly terminal on RP2040 provides an interactive environment for entering and executing assembly programs.
5. **Output Display:** The time-multiplexed LED display successfully shows 8-bit patterns.

The hybrid approach with the RP2040 microcontroller provides a user-friendly interface while the FPGA ensures deterministic, real-time execution. This project serves as a practical

example of CPU design, FPGA implementation, and hardware-software co-design, suitable for both educational and prototyping purposes.

For complete documentation, including the full Verilog source code, assembly terminal script, and a demonstration video, please visit the project's GitHub repository: https://github.com/Devesh-pande2/NOVA-8.

References

1. Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
2. Lattice Semiconductor. (2024). *iCE40 UltraPlus Family Data Sheet*. https://www.latticesemi.com/
3. Raspberry Pi Ltd. (2024). *RP2040 Datasheet*. https://datasheets.raspberrypi.com/
4. HC-SR04 Ultrasonic Sensor Datasheet. (2018). https://www.electroschematics.com/
5. Yosys Open Synthesis Suite. (2024). https://yosyshq.net/yosys/
6. nextpnr – FPGA Place and Route. (2024). https://github.com/YosysHQ/nextpnr
7. IceStorm FPGA Tools. (2024). http://www.clifford.at/icestorm/
8. Go Configure Software Hub User Guide. Lattice Semiconductor.
9. Verilog HDL Specification, IEEE Std 1364-2005.
10. SPI Bus Specification. Motorola Inc.
11. OpenRISC Project. (2024). https://openrisc.io/
12. J1 Forth CPU. (2024). https://github.com/jamesbowman/j1

A Pin Configuration and RTL Diagrams

A.1 FPGA Pin Configuration

Figure 9 shows the pin configuration of the board.

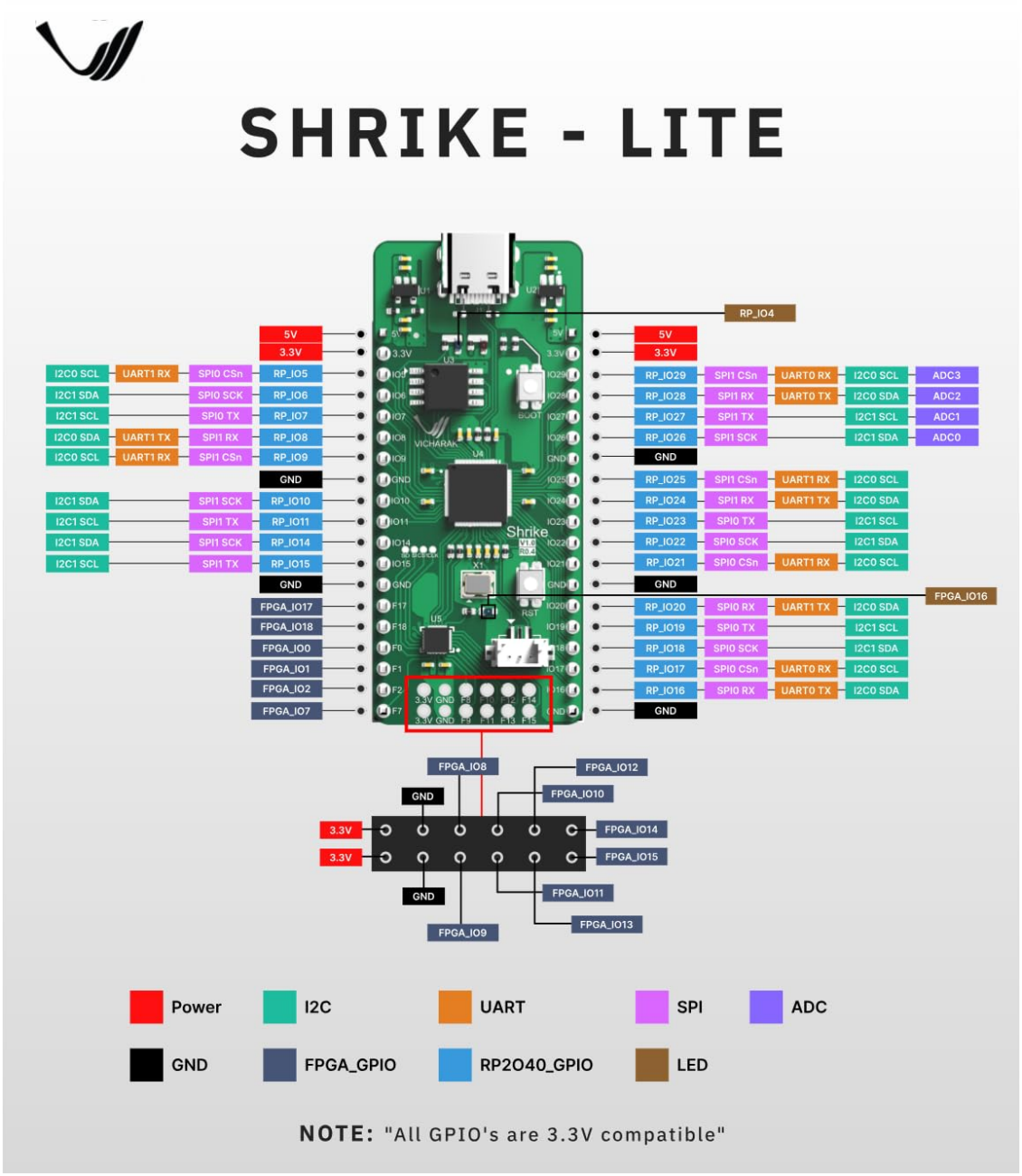

Figure 9: Board Pin Configuration

A.2 RTL Schematic

Figure 10 shows the RTL schematic generated from the synthesis tool.

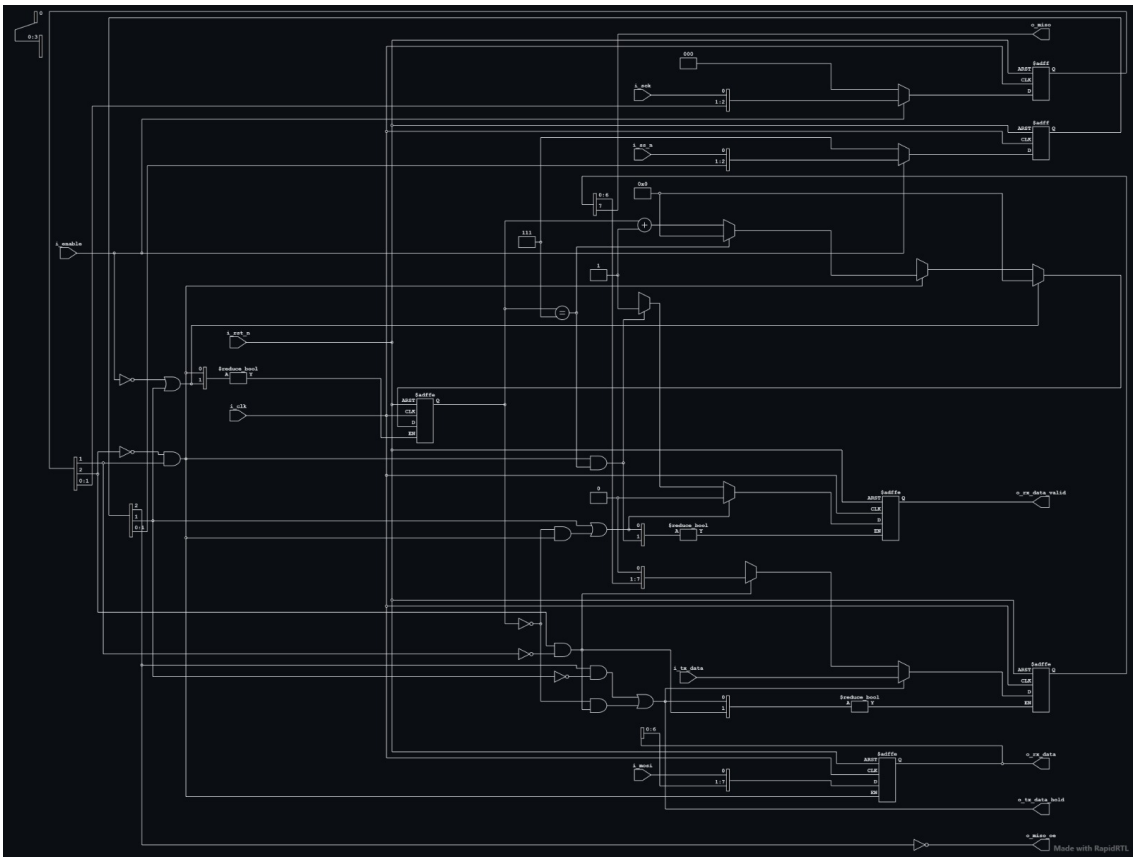

Figure 10: RTL Schematic of the FPGA Design

B Design Constraints

The following constraints were applied during synthesis:

```
# Clock definition
create_clock -name clk -period 20.000 [get_ports clk]

# Input delays
set_input_delay -clock clk 2.000 [get_ports spi_ss_n]
set_input_delay -clock clk 2.000 [get_ports spi_sck]
set_input_delay -clock clk 2.000 [get_ports spi_mosi]
set_input_delay -clock clk 2.000 [get_ports echo]

# Output delays
set_output_delay -clock clk 2.000 [get_ports spi_miso]
set_output_delay -clock clk 2.000 [get_ports trig]
set_output_delay -clock clk 2.000 [get_ports led16]

# False paths (asynchronous)
set_false_path -from [get_ports spi_sck]
set_false_path -from [get_ports spi_ss_n]
```

```
18
19 # Clock uncertainty
20 set_clock_uncertainty 0.500 [get_clocks clk]
```

Listing 1: Timing Constraints (SDC)

C Assembly Program Examples

C.1 Example 1: Simple Addition

```
1 LDA 10
2 ADD 20
3 OUTA
```

Listing 2: Simple Addition Program

Expected Output: LED displays pattern 0x1E (30)

C.2 Example 2: Sensor Controlled Loop

```
1 SENSE
2 JZ 5
3 OUT 0xFF
4 JMP 1
5 OUT 0x00
6 JMP 1
```

Listing 3: Sensor Controlled Program

Expected Output: LED turns ON when object detected, OFF when clear.

C.3 Example 3: Sequence Generation

```
1 LDA 0x01
2 OUTA
3 LDA 0x02
4 OUTA
5 LDA 0x04
6 OUTA
7 LDA 0x08
8 OUTA
```

Listing 4: LED Sequence Program

Expected Output: LED shows blinking pattern cycling through bits.